DESENVOLVIMENTO ANDROID MODERNO — ATIVIDADE AVALIATIVA

Missão: Seu Próprio App, com Navegação de Verdade

Replique a arquitetura de navegação construída em aula — Navigation Compose, rotas type-safe e um fluxo condicional — num app com tema escolhido por você.

Contexto

Em aula, construímos o HabitTracker: um app com Splash condicional, Login, Bottom Navigation com três abas e uma tela de detalhe alimentada por parâmetro type-safe. Nesta atividade, você vai provar que entendeu os quatro padrões de navegação — não apenas copiou o código — construindo a mesma estrutura de telas para um tema diferente.

Especificação da Missão

Escolha **um** tema de app e construa a estrutura de telas abaixo do zero:

Lista de Filmes Favoritos | **Catálogo de Receitas** | **Diário de Treinos**

Qualquer que seja o tema, a estrutura de navegação precisa ser: **Splash → (condicional) → Login ou Painel Principal com Bottom Navigation de 3 abas → uma das abas leva a uma Lista → que leva a uma tela de Detalhe com parâmetro.**

Requisitos Obrigatórios

- **Rotas tipadas:** todas as rotas do app declaradas como uma `sealed interface` com `@Serializable` — nenhuma rota como `String` solta.
- **Navegação Linear:** a Splash leva a outra tela automaticamente, sem interação do usuário.
- **Navegação Condicional:** a Splash decide o destino com base em algum estado (ex: logado/não logado) — a decisão não pode estar dentro da própria Splash, e sim em quem a chama.
- **Parâmetro Type-Safe:** a tela de Lista navega até a tela de Detalhe passando um identificador (ex: `Int` ou `String`) através de uma `data class` de rota — lido de volta com `toRoute()`.
- **Bottom Navigation Persistente:** três abas principais, implementadas com `saveState`, `restoreState` e `launchSingleTop` juntos — trocar de aba não pode resetar a rolagem da lista.
- **ViewModel único e compartilhado:** criado uma vez (no componente que hospeda o `NavHost`) e passado como parâmetro para cada tela — nenhuma tela chama `viewModel()` por conta própria.
- **Paleta própria:** um arquivo de tema com `lightColorScheme(...)` definindo cores específicas do seu app — nenhuma tela usa `Color(0xFF...)` direto.
- **Estado vazio tratado:** a tela de Lista mostra uma mensagem quando não há itens, em vez de aparecer em branco.
- **Acessibilidade básica:** todo `Icon` sem texto visível ao lado tem um `contentDescription` que descreve a ação.

Rubrica de Correção

Critério	Peso	O que será verificado
Rotas type-safe	**20%**	Sealed interface com @Serializable; nenhuma rota como String solta; toRoute() usado corretamente.
Navegação linear e condicional	**20%**	Splash funcional; decisão de destino delegada a quem chama a Splash, não decidida dentro dela.
Bottom Navigation persistente	**25%**	Três abas funcionais; estado da lista preservado ao trocar de aba (saveState/restoreState/launchSingleTop).
Design de verdade	**20%**	Paleta própria via colorScheme; estado vazio tratado; ícones com contentDescription.
Arquitetura (ViewModel único)	**15%**	Uma única instância de ViewModel compartilhada entre todas as telas.

Checklist de Autoavaliação

Antes de entregar, confirme:

- ☐ Nenhuma rota do meu app é uma String solta — todas são classes com @Serializable.
- ☐ Testei trocar de aba, rolar a lista, trocar de novo e voltar — a rolagem continuou onde eu deixei.
- ☐ Toquei em itens diferentes da lista e cada um abriu o detalhe correto (não sempre o mesmo).
- ☐ Apaguei todos os itens (ou testei antes de adicionar algum) e vi a mensagem de lista vazia.
- ☐ Procurei por algum `Color(0xFF...)` solto dentro de uma tela — e não achei nenhum.
- ☐ Rodei o app do zero (cold start) e ele funcionou sem crashar.

Instruções de Entrega

Suba o projeto completo em um repositório público no GitHub e entregue o link na plataforma indicada pelo professor. O `README.md` deve conter: o tema escolhido, as telas implementadas, e como rodar o projeto no Android Studio.